Clustering – Model Selection

Lab Task – 11 & 12

Identify the pattern or group the customer based on K-Means Clustering and Hierarchial Clustering using the following parameters (independent variables):

Note: You must use Mall_Customers.csv

Steps:

1. Import libraries and datasets
2. Find the optimal number of clusters using the elbow method
3. Apply model on the Training set
4. Visualize the cluster

Answer:

K-Means:

a.	Importing the libraries

	`import numpy as np`
	`import matplotlib.pyplot as plt`
	`import pandas as pd`

b.	Importing the dataset

	`dataset = pd.read_csv('Mall_Customers.csv')`
	`X = dataset.iloc[:, [3, 4]].values`
	`print(X)`

c.	Using the elbow method to find the optimal number of clusters

	`from sklearn.cluster import KMeans`
	`wcss = []`
	`for i in range(1, 11):`
	`  kmeans = KMeans(n_clusters = i, init = 'k-means++',`
	`random_state = 42)`
	`  kmeans.fit(X)`
	`  wcss.append(kmeans.inertia_)`
	`plt.plot(range(1, 11), wcss)`
	`plt.title('The Elbow Method')`
	`plt.xlabel('Number of clusters')`
	`plt.ylabel('WCSS')`
	`plt.show()`

d.	Apply the K-means model on the Training set

	`kmeans = KMeans(n_clusters = 5, init = 'k-means++', random_state =`
	`42)`
	`y_kmeans = kmeans.fit_predict(X)`
	`print(y_kmeans)`

e.	Visualise the Cluster

	`plt.scatter(X[y_kmeans == 0, 0], X[y_kmeans == 0, 1], s = 100, c = 'red', label = 'Cluster 1')`
	`plt.scatter(X[y_kmeans == 1, 0], X[y_kmeans == 1, 1], s = 100, c = 'blue', label = 'Cluster 2')`
	`plt.scatter(X[y_kmeans == 2, 0], X[y_kmeans == 2, 1], s = 100, c = 'green', label = 'Cluster 3')`
	`plt.scatter(X[y_kmeans == 3, 0], X[y_kmeans == 3, 1], s = 100, c = 'cyan', label = 'Cluster 4')`
	`plt.scatter(X[y_kmeans == 4, 0], X[y_kmeans == 4, 1], s = 100, c = 'magenta', label = 'Cluster 5')`
	`plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1], s = 300, c = 'yellow', label = 'Centroids')`
	`plt.title('Clusters of customers')`
	`plt.xlabel('Annual Income (k$)')`
	`plt.ylabel('Spending Score (1-100)')`
	`plt.legend()`
	`plt.show()`

Hierarchical Clustering:

a.	Importing the libraries

	`import numpy as np`
	`import matplotlib.pyplot as plt`
	`import pandas as pd`

b.	Importing the dataset

	`dataset = pd.read_csv('Mall_Customers.csv')`
	`X = dataset.iloc[:, [3, 4]].values`
	`print(X)`

c.	Using the dendrogram to find the optimal number of clusters

	`import scipy.cluster.hierarchy as sch`
	`dendrogram = sch.dendrogram(sch.linkage(X, method = 'ward'))`
	`plt.title('Dendrogram')`
	`plt.xlabel('Customers')`
	`plt.ylabel('Euclidean distances')`
	`plt.show()`

d.	Training the Hierarchical Clustering model on the dataset

	`from sklearn.cluster import AgglomerativeClustering`
	`hc = AgglomerativeClustering(n_clusters = 5, affinity = 'euclidean', linkage = 'ward')`
	`y_hc = hc.fit_predict(X)`

e.	Visualise the Cluster

	`plt.scatter(X[y_hc == 0, 0], X[y_hc == 0, 1], s = 100, c = 'red', label = 'Cluster 1')`
	`plt.scatter(X[y_hc == 1, 0], X[y_hc == 1, 1], s = 100, c = 'blue', label = 'Cluster 2')`
	`plt.scatter(X[y_hc == 2, 0], X[y_hc == 2, 1], s = 100, c = 'green', label = 'Cluster 3')`

```
plt.scatter(X[y_hc == 3, 0], X[y_hc == 3, 1], s = 100, c = 'cyan',
label = 'Cluster 4')
plt.scatter(X[y_hc == 4, 0], X[y_hc == 4, 1], s = 100, c =
'magenta', label = 'Cluster 5')
plt.title('Clusters of customers')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.legend()
plt.show()
```

Do it yourself – 1:

Perform K-means clustering analysis on the 'Marks' dataset using 'Quiz', 'Practical', 'Project', and 'Final' marks as features. Determine the optimal number of clusters using the elbow method. Since visualizing clusters in four dimensions is challenging, apply Principal Component Analysis (PCA) to reduce the dimensionality of the data to two or three dimensions. Visualize the clustered data in this reduced space to understand how students are grouped based on their academic performance across the different assessments.

Steps to be followed:

- **Data Preprocessing**:
  - Filled missing values using the column mean.
  - Scaled the data using `StandardScaler` to standardize feature values.
- **Elbow Method**:
  - Plotted inertia values for different numbers of clusters to find the optimal `K`.
- **K-means Clustering**:
  - Applied K-means with the selected number of clusters (`K=3` in the example).
  - Assigned cluster labels to the dataset.
- **Visualization**:
  - Used PCA to reduce data to 2 dimensions for visualization.
  - Plotted the clusters with distinct colors.

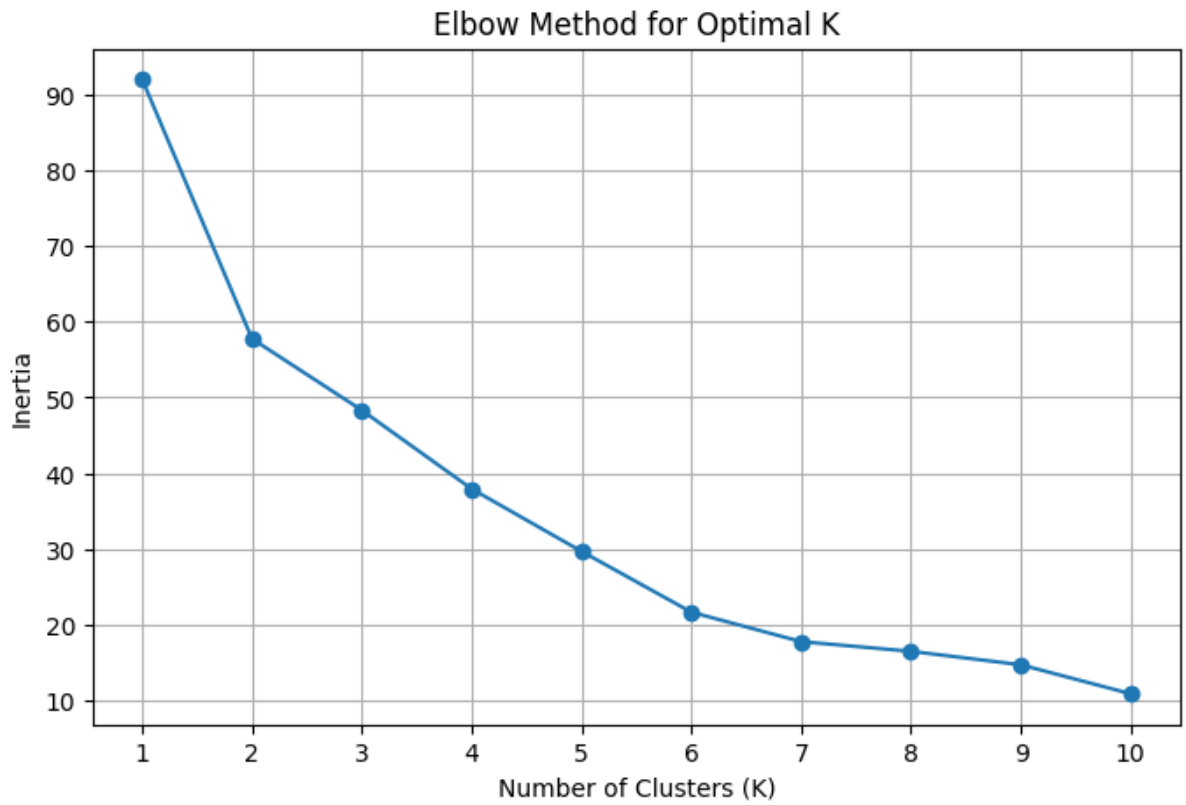
Sample Code:

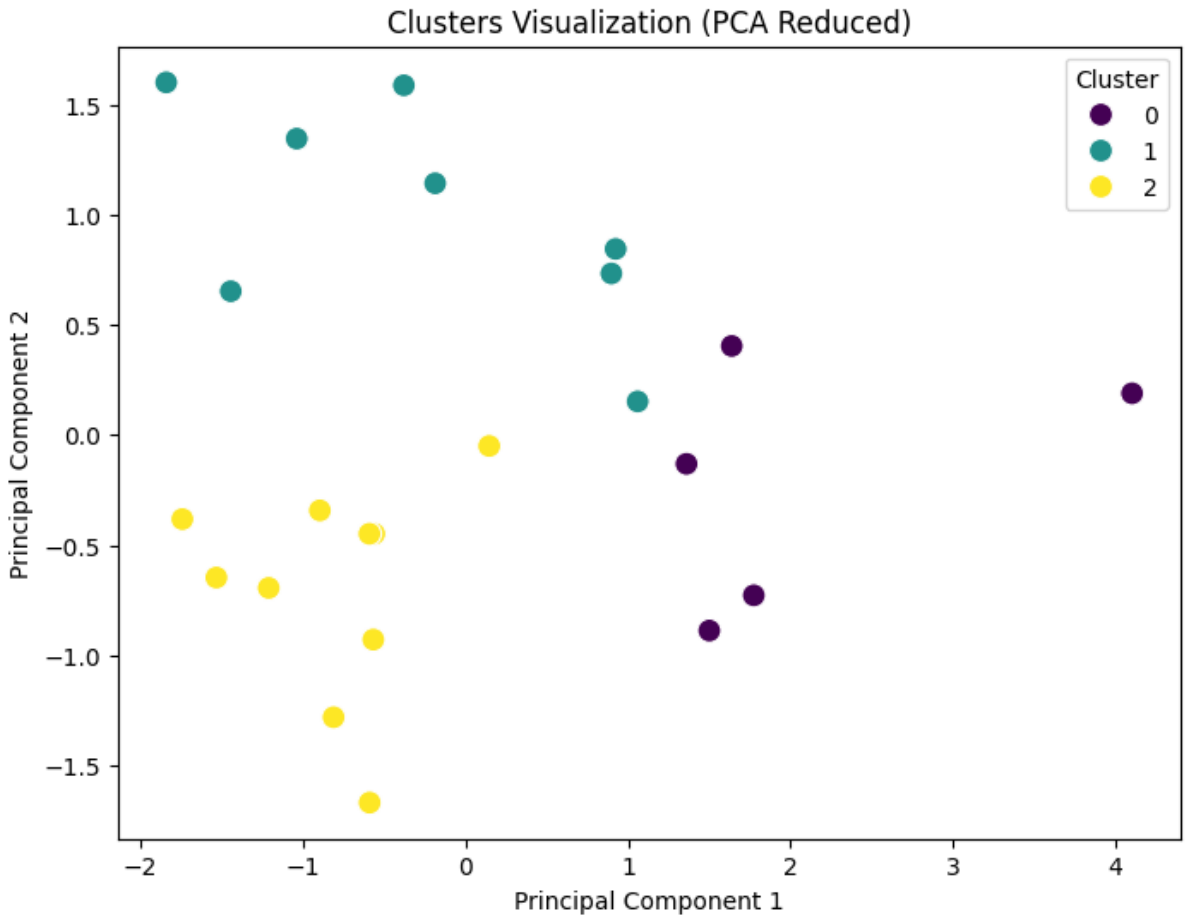
```
# Importing necessary libraries
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import seaborn as sns

# Load the dataset
file_path = 'Marks.csv'  # Update with your dataset
path
data = pd.read_csv(file_path)

# Select relevant features for clustering
data = data[['Quiz', 'Practical', 'Project',
'Final']].copy()
data = data.fillna(data.mean())  # Fill missing values
with column mean

# Standardize the data
scaler = StandardScaler()
data_scaled = scaler.fit_transform(data)

# Determine the optimal number of clusters using the
Elbow Method
inertia = []
k_range = range(1, 11)  # Test for 1 to 10 clusters
for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=1)
    kmeans.fit(data_scaled)
    inertia.append(kmeans.inertia_)

# Visualize the Elbow Curve
plt.figure(figsize=(8, 5))
plt.plot(k_range, inertia, marker='o', linestyle='-')
plt.title("Elbow Method for Optimal K")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Inertia")
plt.xticks(k_range)
plt.grid(True)
plt.show()

# Apply K-means with the chosen number of clusters
(e.g., K=3)
```

```
k = 3  # Replace with the chosen number of clusters
from the elbow plot
kmeans = KMeans(n_clusters=k, random_state=1)
clusters = kmeans.fit_predict(data_scaled)

# Add cluster labels to the original dataset
data['Cluster'] = clusters

# Visualize the clusters using PCA (reduce to 2
dimensions)
pca = PCA(n_components=2)
data_pca = pca.fit_transform(data_scaled)
data['PCA1'] = data_pca[:, 0]
data['PCA2'] = data_pca[:, 1]

plt.figure(figsize=(8, 6))
sns.scatterplot(x='PCA1', y='PCA2', hue='Cluster',
data=data, palette='viridis', s=100)
plt.title("Clusters Visualization (PCA Reduced)")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.legend(title='Cluster')
plt.show()

# Optional: View cluster centers and basic statistics
for each cluster
print("Cluster Centers (Scaled Data):")
print(kmeans.cluster_centers_)
print("\nCluster Statistics:")
print(data.groupby('Cluster').mean())
```

Expected Output:


Elbow Method for Optimal K
Inertia
Number of Clusters (K)
90
80
70
60
50
40
30
20
10
1
2
3
4
5
6
7
8
9
10


Clusters Visualization (PCA Reduced)
Principal Component 2
Principal Component 1
Cluster
0
1
2
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−2
−1
0
1
2
3
4

```
Cluster Centers (Scaled Data):
[[ 0.90140195  1.27986314  1.28066821  0.62697927]
 [-0.44857126 -0.18516538 -0.55992988  0.79200286]
 [-0.09184397 -0.49179926 -0.1923902  -0.94709192]]

Cluster Statistics:
              Quiz  Practical  Project   Final      PCA1      PCA2
Cluster
0        80.800000      86.80   79.920  79.400  2.074311 -0.231181
1        57.750000      71.75   67.225  80.375 -0.251490  1.006405
2        63.840909      68.60   69.760  70.100 -0.835963 -0.689533
```